

0.0. Le programme qui ne fait rien

Le squelette de tout programme : `int main()`, et la boucle `while`.

1. Ce qu'il faut comprendre

`int main()` est le point de départ. Quand la console s'allume, c'est là qu'elle arrive, et nulle part ailleurs. Tout ce que fait le programme s'écrit entre ses accolades `{ }`.

`while (true)` veut dire « tant que vrai » : ce qui est entre ses accolades recommence **sans jamais s'arrêter**. C'est la **boucle de jeu**. Un jeu ne se termine pas tout seul : il attend le joueur.

`image()` attend la prochaine image de l'écran. Il y en a **60 par seconde** : la boucle fait donc un tour tous les sixièmes de seconde.

Ce programme ne montre rien, mais il tourne. Tous les suivants partent de lui : on ajoutera des choses **avant** la boucle (ce qui se fait une fois) et **dedans** (ce qui se fait à chaque image).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **71 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage      28 o ← la console, à l'allumage — la mise en route de la console
  main()          7 o ← démarrage — ta fonction
  VBlank          8 o ← la console, à chaque image — compte les images, 60 fois par seconde
  AttendreImage  28 o ← image() — attend l'image suivante
Lettres gravées : aucune
Pas gravé (jamais appelé) : AvancerAirs, AttendreVBlank, AttendreAcces, AttendreFinVBlank, AdresseCase,
EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins,
InstallerTransfert, CopierTuiles, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, LireManette, Tuiles,
DonneesTransfert, Notes
Fonctions du langage appelées :
  image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

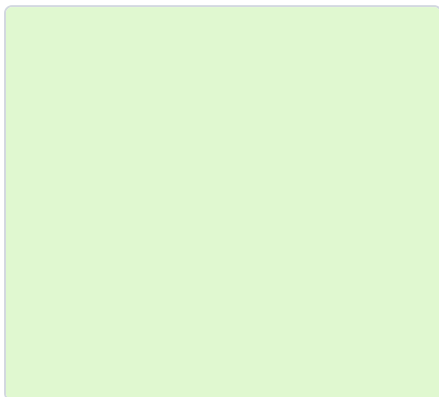
```
// Tout ce qui suit deux barres « // » est un COMMENTAIRE :
// la console ne le lit pas, il est là seulement pour toi.
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {           // main = « principal » : quand la console s'allume, elle arrive ICI.
                        // Tout le programme s'écrit entre cette accolade ouvrante
                        // et la dernière accolade fermante.
    while (true) {      // while (true) = « tant que vrai » : ce qui est entre les
                        // accolades recommence pour TOUJOURS. C'est la boucle de jeu.
        image();         // Attend la prochaine image de l'écran (60 par seconde).
                        // Le « ; » termine l'instruction, comme un point en fin de phrase.
    }                   // Fin de la boucle : on repart à image().
}                       // Fin de main. Ce programme ne montre rien, mais il tourne.
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

4. Ce qu'on doit voir

Rien : l'écran reste vide, mais la console tourne.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ l'écran est allumé

✓ et rien n'y est écrit

Ensuite : **0.0.1. Pourquoi écrire « `#include` » ?** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++